

Course Code: **IDC 222**



Scientific Computing & Data Visualization

Reji J

School of Data Science

IISER Thiruvananthapuram

Templates in C++



Add two numbers (int, float)

```
#include<iostream>
```

```
using namespace std;
```

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
float add(float a, float b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
int main()
```

```
{
```

```
    int x = 10;
```

```
    int y = 20;
```

```
    float a = 20.20;
```

```
    float b = 10.20;
```

```
    cout << add(x, y) << endl;
```

```
    cout << add(a, b) << endl;
```

```
    return 0;
```

```
}
```



//without using templates

```
#include<iostream>
using namespace std;

int add(int a, int b)
{
    return a + b;
}

float add(float a, float b)
{
    return a + b;
}

int main()
{
    int x = 10;
    int y = 20;
    float a = 20.20;
    float b = 10.20;

    cout << add(x, y) << endl;
    cout << add(a, b) << endl;

    return 0;
}
```

//with using function templates

```
#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b)
{
    return a + b;
}

int main()
{
    int x = 10;
    int y = 20;

    float a = 20.20;
    float b = 10.20;

    cout << add<int>(x, y) << endl;
    cout << add<float>(a, b) << endl;

    return 0;
}
```



Generic Programming in C++ (Templates)

- **Generic Programming** is a technique that **allows data types** (int, float, string, etc.) and **user-defined types** to be used as parameters in functions and classes.
- It helps in writing **reusable and efficient code**.
- The same algorithm can work with **multiple data types** without rewriting code.
- This avoids creating separate functions or classes for each data type (int, char, string, etc.).

Templates in C++

- **Templates** are a powerful feature of C++ used to implement **Generic Programming**.
- They allow us to write **one function or class** that works with **different data types**.
- The data type is decided **at compile time**.



Types of Templates

- C++ supports **two types** of templates:

1. Function Templates

2. Class Templates

How Templates Work?

- First, we declare a template.
- Immediately after the template declaration, we define a function or class.
- The function or class uses template parameters as data types.
- When the program runs, the compiler creates the required version based on the data type used.



1. Function Template

Defining a Function Template

- A **function template** starts with the keyword **template**.
- Template parameters are written inside **angle brackets** **< >**.
- After the template declaration, the **function definition** is written normally.

Syntax:

```
template <typename T>
T functionName(T parameter1, T parameter2, ...)
{
    // function body
}
```

Example:

```
template <typename T>
T add(T a, T b)
{
    return a + b;
}
```

- **T** is a **template parameter**.
- It represents a **data type** such as **int, float, char, etc.**
- **typename** is a keyword used to declare a **generic data type**.
- When a value is passed to **functionName()**, the compiler automatically creates a separate version of the function for that data type



Contd...

Calling a Function Template

Syntax:

```
functionName<dataType>(parameter1, parameter2, ...);
```

Example:

```
add<int>(10, 20);  
add<float>(10.5, 20.5);
```

- **<float>** explicitly tells the compiler that **T** should be treated as **float**.
- The compiler generates a version of the function using the specified data type.

add<int> → creates a version of **add()** that works with **int**

add<float> → creates a version of **add()** that works with **float**



Function call based on data types

```
#include<iostream>
```

```
template<typename T>
```

```
T add(T num1, T num2) {  
    return (num1 + num2);  
}
```

```
int main() {
```

```
    ... ..
```

```
    result1 = add<int>(2,3);
```

```
    ... ..
```

```
    result2 = add<double>(2.2,3.3);
```

```
    ... ..
```

```
}
```

```
int add(int num1, int num2) {  
    return (num1 + num2);  
}
```

```
double add(double num1, double num2) {  
    return (num1 + num2);  
}
```



Adding Two Numbers Using Function Templates

```
#include <iostream>
using namespace std;

template <typename T>
T add(T num1, T num2)
{
    return (num1 + num2);
}

int main()
{
    int result1;
    double result2;
    // calling with int parameters
    result1 = add<int>(2, 3);
    cout << "2 + 3 = " << result1 << endl;

    // calling with double parameters
    result2 = add<double>(2.2, 3.3);
    cout << "2.2 + 3.3 = " << result2 << endl;

    return 0;
}
```

Output:

2 + 3 = 5

2.2 + 3.3 = 5.5



Exercise 1: Simple Calculator (Template Based)

Create function templates for:

- `add`
- `subtract`
- `multiply`
- `divide`

Use them for both `int` and `float`.

```

#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b)
{
    return a + b;
}

template <typename T>
T sub(T a, T b)
{
    return a - b;
}

template <typename T>
T mul(T a, T b)
{
    return a * b;
}

template <typename T>
T divi(T a, T b)
{
    return a / b;
}

int main()
{
    int x, y;
    float a, b;

    cout << "Enter two integers: ";
    cin >> x >> y;

    cout << "Addition (int): " << add<int>(x, y) << endl;
    cout << "Subtraction (int): " << sub<int>(x, y) << endl;
    cout << "Multiplication (int): " << mul<int>(x, y) << endl;
    cout << "Division (int): " << divi<int>(x, y) << endl;

    cout << "\nEnter two floating-point numbers: ";
    cin >> a >> b;

    cout << "Addition (float): " << add<float>(a, b) << endl;
    cout << "Subtraction (float): " << sub<float>(a, b) << endl;
    cout << "Multiplication (float): " << mul<float>(a, b) << endl;
    cout << "Division (float): " << divi<float>(a, b) << endl;

    return 0;
}

```



2. Class Template

Defining a Class Template

- A **class template** starts with the keyword **template**.
- Template parameters are written inside **angle brackets** **< >**.
- After the template declaration, the **class definition** is written normally.

Syntax:

```
template <class T>
class className
{
    private:
        T var;                // member variable of type T

    public:
        T functionName(T arg); // member function using type T
};
```

Example:

```
template <class T>
class Number
{
    private:
        T num;

    public:
        T getNum();
};
```

- **class** (or **typename**) is a keyword used to **define a template parameter**.
- Inside the class:
 - **Member variables** can be of type **T**
 - **Member functions** can use type **T**



Contd...

Creating a Class Template Object

- After defining a class template, we can create objects **inside** `main()`.
- The data type must be specified **explicitly** while creating the object.

Syntax:

```
className<dataType> objectName;
```

Example:

```
Number<int> obj1;  
Number<float> obj2;  
Number<string> obj2;
```



//C++ program to demonstrate the use of class templates (Without Constructor)

```
#include<iostream>
using namespace std;

template <class T>
class Number
{
    private:
        T num;

    public:
        void setNum(T n)
        {
            num = n;
        }

        T getNum()
        {
            return num;
        }
};
```

```
int main()
{
    Number<int> n1;
    n1.setNum(10);
    cout << n1.getNum() << endl;

    Number<float> n2;
    n2.setNum(25.5);
    cout << n2.getNum() << endl;

    return 0;
}
```



//C++ program to demonstrate the use of class templates (With Constructor)

```
#include<iostream>
using namespace std;

template <class T>
class Number
{
    private:
        T num;

    public:
        Number(T n)
        //parameterized constructor
        {
            num = n;
        }

        T getNum()
        {
            return num;
        }
};
```

```
int main()
{
    Number<int> n1(10);
    cout << n1.getNum() << endl;

    Number<float> n2(25.5);
    cout << n2.getNum() << endl;

    return 0;
}
```





Exercise 1: Template-Based Calculator Using Class Templates

Create a **class template** named **Calculator** that that performs arithmetic operations on two numbers.

Calculator<T>	
-	num1 : T
-	num2 : T
-	Calculator(n1 : T, n2 : T)
-	add() : T
-	subtract() : T
-	multiply() : T
-	divide() : T
-	displayResult() : void

OUTPUT FORMAT:

Int results:

Numbers: 2 and 1

2 + 1 = 3

2 - 1 = 1

2 * 1 = 2

2 / 1 = 2

Float results:

Numbers: 2.4 and 1.2

2.4 + 1.2 = 3.6

2.4 - 1.2 = 1.2

2.4 * 1.2 = 2.88

2.4 / 1.2 = 2

Note: Use a **parameterized constructor** to initialize the data members.

```
//Sample Calculator using class templates
```

```
#include <iostream>
```

```
using namespace std;
```

```
template <class T>
```

```
class Calculator
```

```
{
```

```
private:
```

```
    T num1, num2;
```

```
public:
```

```
    Calculator(T n1, T n2) //Constructor
```

```
{
```

```
    num1 = n1;
```

```
    num2 = n2;
```

```
}
```

```
void displayResult()
```

```
{
```

```
    cout << "Numbers: " << num1 << " and " << num2;
```

```
    cout << num1 << " + " << num2 << " = " << add();
```

```
    cout << num1 << " - " << num2 << " = " << subtract();
```

```
    cout << num1 << " * " << num2 << " = " << multiply();
```

```
    cout << num1 << " / " << num2 << " = " << divide();
```

```
}
```

```
T add()
```

```
{
```

```
    return num1 + num2;
```

```
}
```

```
T subtract()
```

```
{
```

```
    return num1 - num2;
```

```
}
```

```
T multiply()
```

```
{
```

```
    return num1 * num2;
```

```
}
```

```
T divide()
```

```
{
```

```
    return num1 / num2;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    Calculator<int> intCalc(2, 1);
```

```
    Calculator<float> floatCalc(2.4, 1.2);
```

```
    cout << "Int results:" << endl;
```

```
    intCalc.displayResult();
```

```
    cout << "Float results:";
```

```
    floatCalc.displayResult();
```

```
    return 0;
```

```
}
```



Exercise 2: Statistics Calculator (Class Template)

Create a Statistics Calculator that computes the **maximum**, **minimum**, and **average** of two numbers using class templates.

StatisticsCalculator<T>

- num1 : T
- num2 : T
- StatisticsCalculator(n1 : T, n2 : T)
- getMax() : T
- getMin() : T
- getAverage() : T
- displayResult() : void

OUTPUT FORMAT:

Integer Statistics:

Numbers: 10 and 20

Maximum: 20

Minimum: 10

Average: 15

Float Statistics:

Numbers: 5.5 and 8.2

Maximum: 8.2

Minimum: 5.5

Average: 6.85

Note: Use a **parameterized constructor** to initialize the data members.

```

#include <iostream>
using namespace std;

template <class T>
class StatisticsCalculator
{
private:
    T num1, num2;

public:
    StatisticsCalculator(T n1, T n2)
    {
        num1 = n1;
        num2 = n2;
    }

    void displayResult()
    {
        cout << "Numbers: " << num1 << "and" << num2;
        cout << "Maximum: " << getMax() << endl;
        cout << "Minimum: " << getMin() << endl;
        cout << "Average: " << getAverage() << endl;
        cout << "-----" << endl;
    }
}

```

```

T getMax()
{
    return (num1 > num2) ? num1 : num2;
}

T getMin()
{
    return (num1 < num2) ? num1 : num2;
}

T getAverage()
{
    return (num1 + num2) / 2.0;
}

};

int main()
{
    StatisticsCalculator<int> intStats(10, 20);
    StatisticsCalculator<float> floatStats(5.5, 8.2);

    cout << "Integer Statistics:" << endl;
    intStats.displayResult();

    cout << "Float Statistics:" << endl;
    floatStats.displayResult();

    return 0;
}

```



Introduction to C++ STL



Introduction to STL (Standard Template Library)

- **STL** stands for **Standard Template Library**.
- It is a collection of **template-based classes and functions** in C++.
- STL provides ready-made implementations of:
 - common **data structures**
 - common **algorithms**

What Does STL Provide?

- **Data structures** such as:
 - **arrays**
 - **lists**
 - **stacks**
 - **queues**
 - **vectors**
- **Algorithms** such as:
 - **sorting**
 - **searching**
 - **inserting**
 - **deleting**



Why STL?

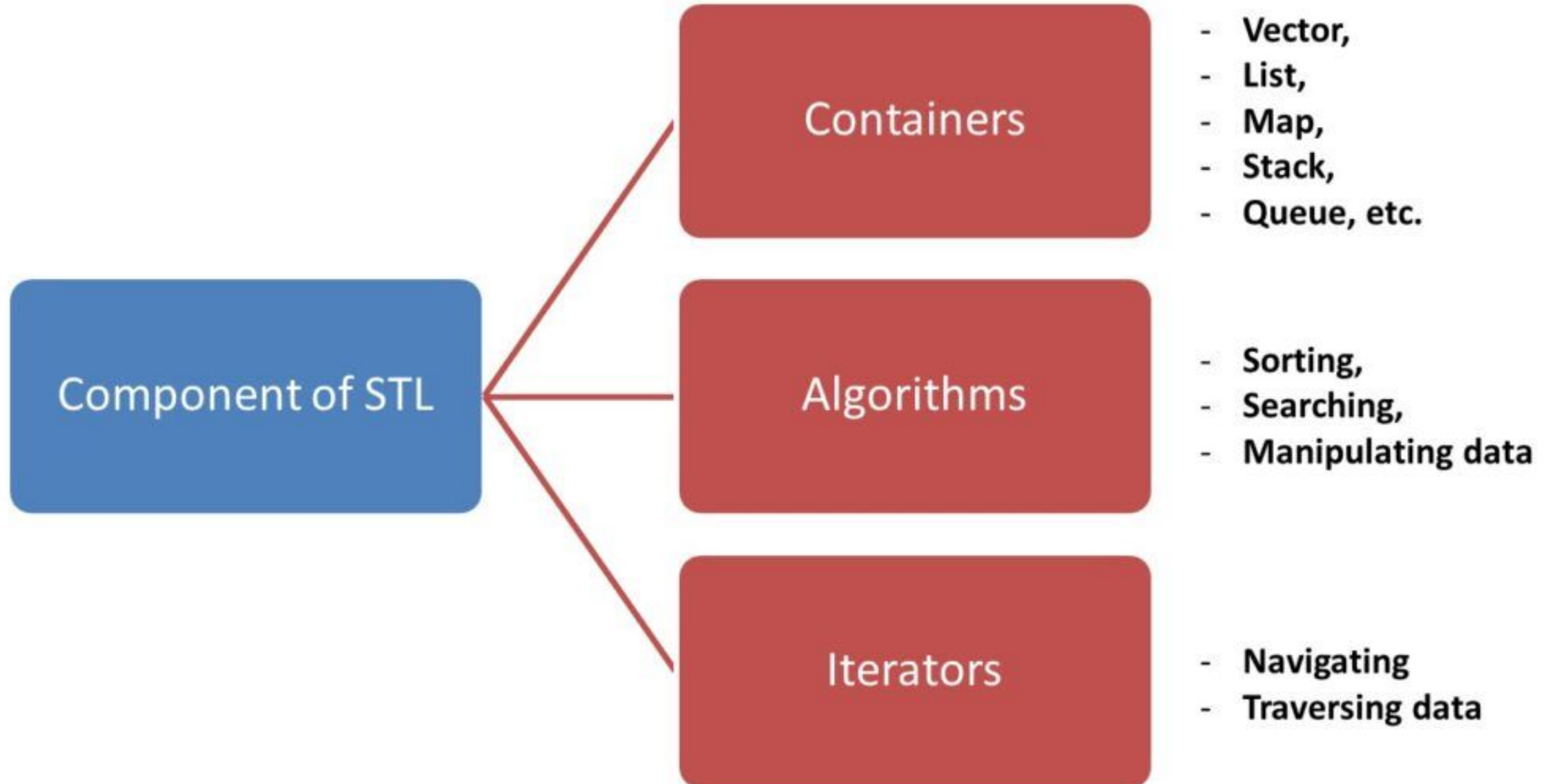
- STL is a **generalized library**.
- It works with **almost all data types** using templates.
- Avoids writing the same implementation again and again.
- Improves **code reusability and efficiency**.
- STL helps in handling **large amounts of data** efficiently.

STL and Data Structures

- A **data structure** is used to store and organize data efficiently.
Example: An **array** stores multiple elements in a single variable.
- **C++ provides many data structures through STL** to handle data in different ways.
- Choosing the right data structure and algorithm makes programs:
 - faster
 - more efficient
 - easier to maintain



Components of STL



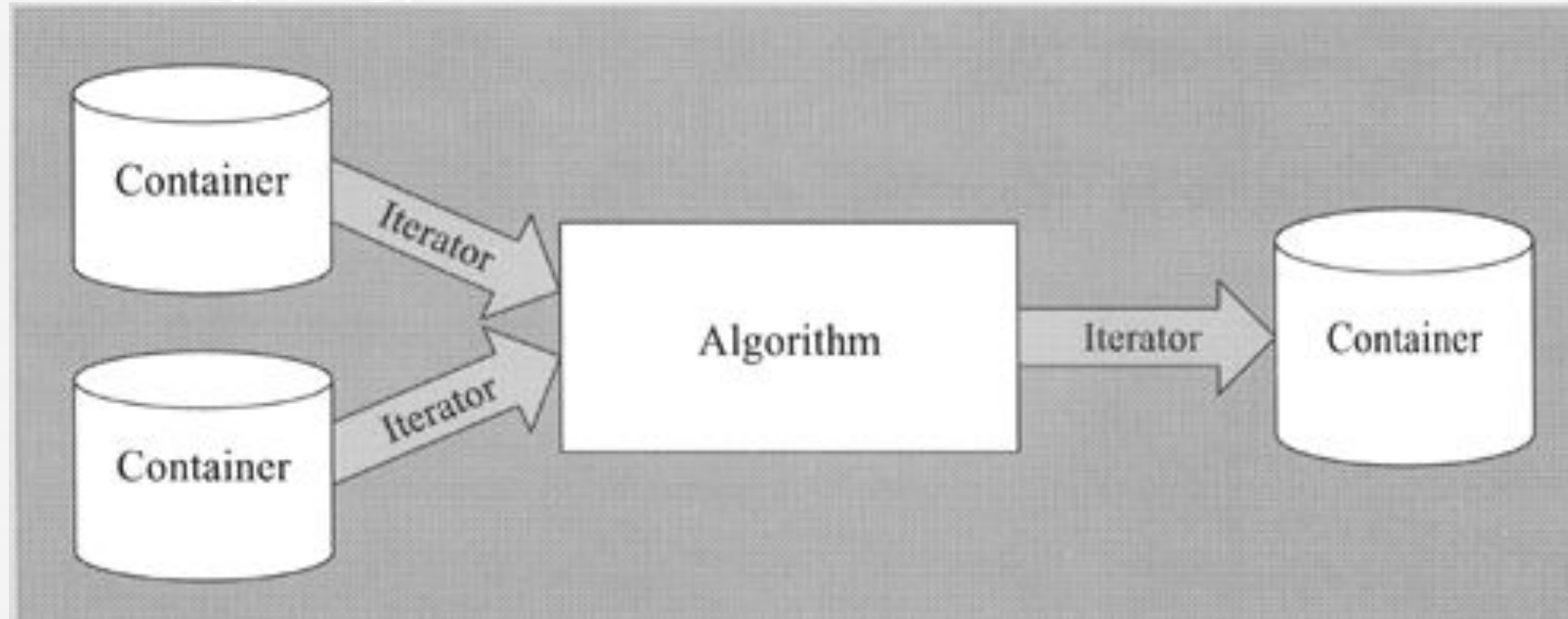
Components of STL

- There are three major components of the C++ Standard Template Library (STL):

1) **Containers**

2) **Iterators**

3) **Algorithms**



Containers

- **Containers** are data structures used to **store data**.
- **Examples include: vector, list, deque, set, map.**
- Each container is implemented as a **template class**.
- Containers provide **member functions** to perform basic operations such as:
 - a. **insertion**
 - b. **deletion**
 - c. **accessing elements**
- Every STL container is defined in its **own header file**.

Types of Containers

Array-based containers

Example: **array, vector**

Linked list-based containers

Example: **list, forward_list**

Key-value containers

Example: **map, unordered_map**



Algorithms

- **Algorithms** are **functions** that **perform operations on data** stored in containers.
- **Common STL algorithms include:** **sort()**, **find()**, **count()** **etc.**
- Algorithms work on **containers** through **iterators**.
- STL algorithms provide **efficient implementations** for tasks such as:
 - sorting**
 - searching**
 - modifying**
 - manipulating data**
- Most STL algorithms are defined in:
 - <algorithm>**
 - <numeric>**



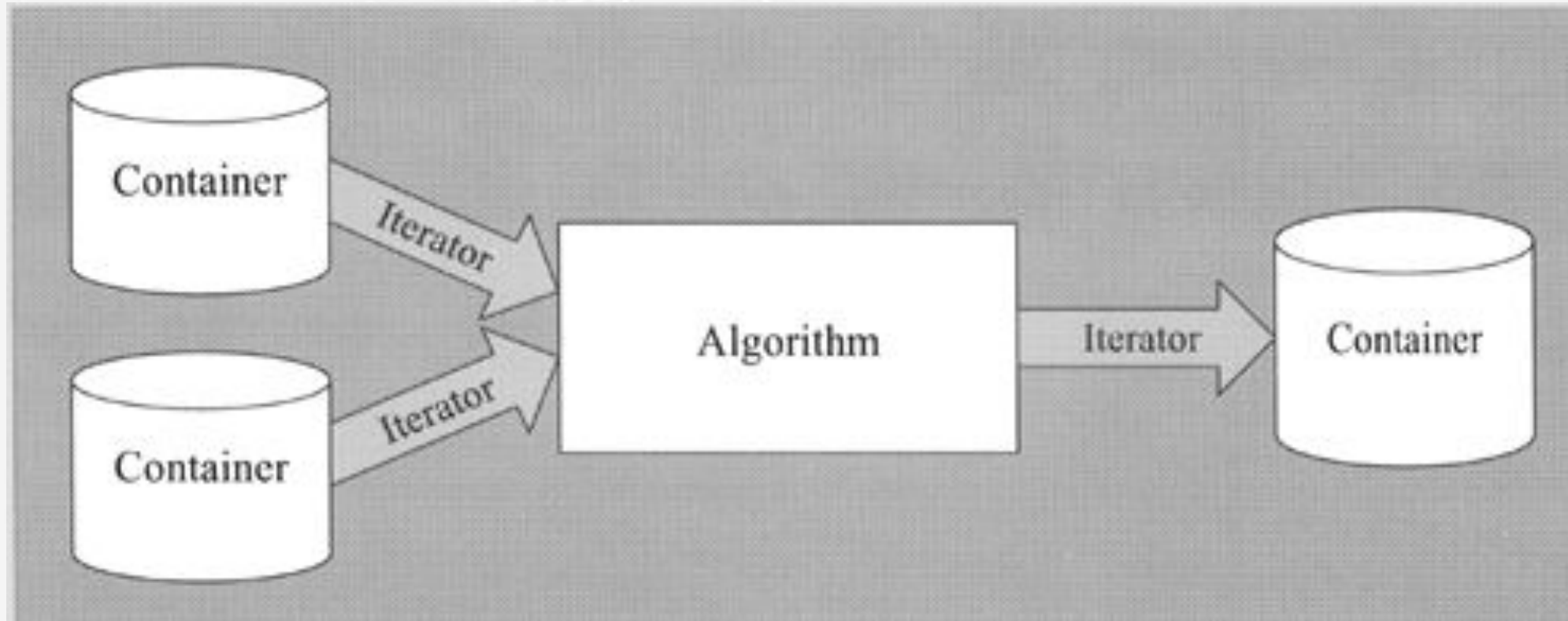
Iterators

- **Iterators** are objects used to **access elements** of a container.
- They behave like **pointers**.
- Iterators point to the **memory locations** of container elements.
- They act as a **bridge** between **containers** and **algorithms**.
- STL algorithms use iterators to perform operations on stored data.
- Iterators behave similar to pointers:
 - ++** → moves the iterator to the next element
 - *** → accesses the value pointed to by the iterator



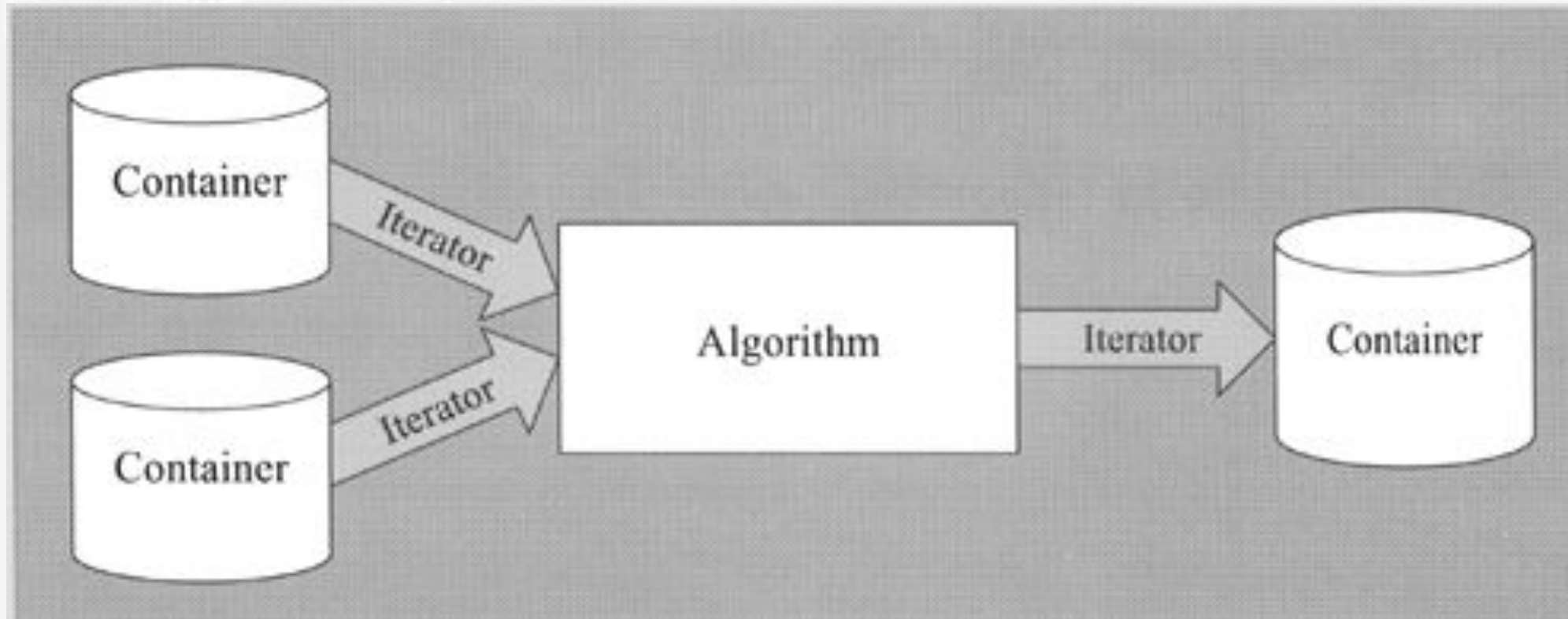
Core Concept of STL

- The design of STL is based on the **separation of data and operations**.
- **Data** is managed by **container classes**.
- **Operations** are provided by **algorithms**.
- **Iterators** act as the **connecting link (glue)** between containers and algorithms.



How STL Works?

- Containers store the **data**.
- Algorithms perform **operations**.
- **Iterators** allow **algorithms** to access and traverse container elements.



Containers

```
graph TD; Containers[Containers] --> Sequence[Sequence Container]; Containers --> Associative[Associative container]; Containers --> Unordered[Unordered Associative Containers]; Containers --> Adapters[Container Adapters]; Sequence --> Array[• Array]; Sequence --> Vector[• Vector]; Sequence --> Deque[• Deque]; Sequence --> List[• List]; Sequence --> ForwardList[• Forward List]; Associative --> Set[• Set]; Associative --> Multiset[• Multiset]; Associative --> Map[• Map]; Associative --> Multimap[• Multimap]; Unordered --> UnorderedSet[• Unordered Set]; Unordered --> UnorderedMultiset[• Unordered Multiset]; Unordered --> UnorderedMap[• Unordered Map]; Unordered --> UnorderedMultimap[• Unordered Multimap]; Adapters --> Stack[• Stack]; Adapters --> Queue[• Queue]; Adapters --> PriorityQueue[• Priority Queue];
```

Sequence Container

- Array
- Vector
- Deque
- List
- Forward List

Associative container

- Set
- Multiset
- Map
- Multimap

Unordered Associative Containers

- Unordered Set
- Unordered Multiset
- Unordered Map
- Unordered Multimap

Container Adapters

- Stack
- Queue
- Priority Queue

Types of Containers in STL

- STL containers can be classified into four types:

1. Sequence Containers ✓
2. Associative Containers ✓
3. Container Adaptors
4. Unordered Associative Containers

👉 *In this course, we mainly focus on the first two types.*



Sequence Containers

- Sequence containers store elements in a **linear order**.
- Elements are accessed **sequentially using their position**.
- Internally implemented using **Arrays** and **Linked lists**
- They allow iteration from the first element to the last element.
- There are four **main sequence containers**:

1. **Vector**

2. **Array**

3. **Deque**

4. **List**



Vector



Vector

- **Vector** is a **dynamic array** in C++ STL.
- It **automatically resizes** when elements are added or removed.
- Unlike fixed-size arrays, vectors can **grow or shrink at runtime**.
- The container handles **memory allocation and deallocation internally**.

Memory Layout & Access

- Elements are stored in **contiguous (adjacent) memory locations**.
- Provides **fast random access** to elements.
- Supports: **Subscript operator []** and **Iterators** for traversal (access elements).

Insertion & Reallocation

- Elements are usually **inserted at the end** of the vector.
- When capacity is exceeded:
 - Vector **allocates a larger memory block**
 - Copies existing elements to the new location (**reallocation**)



Vector (C++ STL)

- Vector is defined as the **std::vector class template**.
- It is available in the **<vector> header file**.
- Vectors are used to **store elements of the same data type**.
- Unlike arrays, **vectors are dynamic** in nature.
- The **size of a vector can change during program execution** as required.

Why Use Vector?

- No fixed size at declaration
- Automatic memory management
- Easy insertion and deletion of elements
- Faster access compared to linked lists



Vector - Header File & Syntax

Using Vector in C++

To use vectors, include the header file:

```
#include <vector>
```

Vector Declaration

Syntax: `vector<T> vector_name;`

Example: `vector<int> numbers;`
`vector<float> marks;`
`vector<string> fruits;`

- `<T>` specifies the data type of the vector.
- It can be: `int char float double`



Vector Initialization

- There are different ways to initialize a vector in C++.
- **Method 1:** Initialization using **initializer list**

```
vector<int> vector1 = {1, 2, 3, 4, 5};  
vector<int> vector2 {1, 2, 3, 4, 5};
```

- **Method 2:** Initialization using **size** and **value**

```
vector<int> vector3(5, 12);
```

Equivalent to:

```
vector<int> vector3 = {12, 12, 12, 12, 12};
```

- Values are **directly provided** during initialization.
- Both `vector1` and `vector2` contain: 1 2 3 4 5
- 5 → size of the vector
- 12 → value to initialize each element
- Creates a vector of size 5
- All elements are initialized with 12



Accessing Vector Elements

1) Using index (like array)

```
vector<int> v = {5, 10, 15};  
  
cout << v[0];    // Output: 5  
cout << v[1];    // Output: 10
```

2) Using `at()` function

```
vector<int> v = {5, 10, 15};  
  
cout << v.at(0);  // Output: 5  
cout << v.at(2);  // Output: 15
```

- `at()` provides bounds checking
- `[ ]` is faster but unsafe if index is invalid



Printing Vector Elements Using for Loop

1) Using Index-Based (for loop)

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v = {10, 20, 30, 40};

    for (int i = 0; i < v.size(); i++)
    {
        cout << v[i] << endl;

        ---OR---

        cout << v.at(i) << endl;
    }
    return 0;
}
```

- **size()** returns the number of elements in the vector
- **[]** operator is used for direct access
- **at()** also used to access elements
- Similar to array traversal



Range-Based for Loop (for-each Loop)

- Also called a **range-based for loop** or **for-each loop**
- Simplifies iteration over containers such as: **vector**, **array**, **list**, **set**, etc.
- Automatically iterates over **each element** in the container
- Eliminates the need for index or iterator handling

Syntax:

```
for (data_type variable : container)
{
    // Code to execute
}
```

Using auto Keyword

```
for (auto variable : container)
{
    // Code to execute
}
```

Note: **auto** automatically deduces the data type from the container elements



Printing Vector Elements Using Range-Based for Loop

2) Using Range-Based (for each Loop)

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main()
{
    vector<string> fruits = {"Apple", "Banana", "Mango"};

    for (string f : fruits) //auto also we can use here
    {
        cout << f << endl;
    }

    return 0;
}
```

- Simple and readable
- No index handling required



Important Vector Functions

Function	Purpose
<code>push_back(x)</code>	Add element at the end
<code>pop_back()</code>	Remove last element
<code>size()</code>	Returns number of elements
<code>empty()</code>	Checks if vector is empty
<code>clear()</code>	Removes all elements
<code>front()</code>	Access first element
<code>back()</code>	Access last element



Sample Program

```
#include <vector>
#include <iostream>
using namespace std;

-----

int main ()
{
    vector<int> v {2, 7, 9};
    cout << v.size() << '\n';

    -----

    cout << "1st Element" << v[0] << '\n';
    cout << "2nd Element" << v[1] << '\n';

    -----

    v[1] = 4;
    cout << "Reassigned" << v[1] << '\n';

    -----

    cout << "1st Element" << v.front() << '\n';
    cout << "last Element" << v.back() << '\n';
}
```

header files

Initialize with 3 numbers

3 (number of elements)

2

7

assign new value

4

2 (first element)

9 (Last element)

⚠ Careful!

vector<int> v1 {5,2}; →

5	2
---	---

vector<int> v2 (5,2); →

2	2	2	2	2
---	---	---	---	---

↑ number of elements ↑ default element value



Appending Elements

```
#include <iostream>
#include <vector>
using namespace std;

-----

int main ()
{
    vector<int> v;
    cout << v.size() << '\n';

    -----

    v.push_back(2);
    cout << v.size() << '\n';

    -----

    v.push_back(7);
    cout << v.size() << '\n';

    -----

    v.push_back(9);
    cout << v.size() << '\n';

    cout << "first element" << v.front() << endl;
    cout << "last element" << v.back() << endl;
}
```



0



1



2



3



Erasing elements(at the end)

`vector<T>::pop_back()`

`vector<T>::clear()`

```
std::vector<int> v {1,2,3,4,5,6};  
cout << v.size() << '\n';
```



1	2	3	4	5	6
---	---	---	---	---	---

6

```
.....  
v.pop_back();  
cout << v.size() << '\n';
```

1	2	3	4	5	
---	---	---	---	---	--

5

```
.....  
v.pop_back();  
cout << v.size() << '\n';
```

1	2	3	4		
---	---	---	---	--	--

4

```
.....  
v.clear();  
cout << v.size() << '\n';
```

--	--	--	--	--	--

0

Program: Input Elements to Vector

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v;
    int n, x;

    cout << "Enter number of elements: ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        cin >> x;
        v.push_back(x);
    }

    cout << "Vector Size : " << v.size() << endl;
    cout << "first element : " << v.front() << endl;
    cout << "last element : " << v.back() << endl;

    // Check if vector is empty
    if (v.empty())
        cout << "\nVector is empty" << endl;
    else
        cout << "\nVector is not empty" << endl;

    return 0;
}
```

- **empty()** function checks whether the vector contains any elements.
- Returns true (**1**) → if the vector has no elements
- Returns false (**0**) → if the vector has one or more elements




```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v;
    int n, x;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> x;
        v.push_back(x);
    }
    cout << "first element : " << v.front() << endl;
    cout << "last element : " << v.back() << endl;
}

```

```

v.pop_back();

cout << "\nAfter pop_back(): ";
for (int x : v)
{
    cout << x << " ";
}
cout << "Vector Size : " << v.size() << endl;

if (v.empty())
    cout << "\nVector is empty" << endl;
else
    cout << "\nVector is not empty" << endl;
return 0;
}

```



Exercise 1: Online Shopping Cart (Using Vector)

Write a C++ program to implement a menu-driven Online Shopping Cart using **vector**.

1. Add item prices to the cart
2. Remove the last added item
3. Display the total number of items and total bill amount
4. Exit the program

Use the following vector functions:

`push_back()`, `pop_back()`, `size()`

OUTPUT FORMAT:

1. Add Item
2. Remove Last Item
3. Display Cart Details
4. Exit

```
Enter your choice: 1
Enter item price: 100
Item added.
```

```
Enter your choice: 1
Enter item price: 200
Item added.
```

```
Enter your choice: 3
Items in Cart: 2
Total Bill Amount: 300
```

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<float> cart;
    int choice;
    float price, total;

    do
    {
        cout << "\n1. Add Item";
        cout << "\n2. Remove Last Item";
        cout << "\n3. Display Cart Details";
        cout << "\n4. Exit";
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                cout << "Enter item price: ";
                cin >> price;
                cart.push_back(price);
                cout << "Item added.";
                break;

            case 2:
                cart.pop_back();
                cout << "Last item removed.";
                break;

```

```

            case 3:
                total = 0;
                for (int i = 0; i < cart.size(); i++)
                {
                    total = total + cart[i];
                }
                cout << "Items in Cart: " << cart.size() << endl;
                cout << "Total Bill Amount: " << total << endl;
                break;

            case 4:
                cout << "Exit the program";
                break;

            default:
                cout << "Invalid choice!";
        }
    } while (choice != 4);

    return 0;
}

```

